Database – No SQL – Mongodb

Session 10 –

1. **Create a MongoDB collection named 'restaurants' and insert 5 documents, each representing a different restaurant with fields: name, cuisine, location, and average_rating.**

Ans:

```
db.createCollection("restaurants")

db.restaurants.insertMany([
    {
        name: "Spice Garden",
        cuisine: "Indian",
        location: "Ahmedabad",
        average_rating: 4.5
    },
    {
        name: "Pizza Paradise",
        cuisine: "Italian",
        location: "Surat",
        average_rating: 4.6
    },
    {
        name: "Dragon Palace",
        cuisine: "Chinese",
        location: "Vadodara",
        average_rating: 4.3
    },
    {
        name: "Burger Hub",
        cuisine: "American",
        location: "Ahmedabad",
        average_rating: 4.2
    },
    {
        name: "Sushi World",
        cuisine: "Japanese",
        location: "Surat",
        average_rating: 4.7
    }
])
```

Verify:

db.restaurants.find().pretty()

2. **Design an 'orders' collection for a food delivery app (like Zomato or Swiggy) where each order document includes: user_id, restaurant_id, items (array of objects with item_name and price), total_amount, and order_date. Insert 3 sample orders.**

Ans:

```
db.createCollection("orders")

db.orders.insertMany([
    {
        user_id: 101,
        restaurant_id: 1,
        items: [
            { item_name: "Butter Chicken", price: 280 },
            { item_name: "Naan", price: 60 }
        ],
        total_amount: 340,
        order_date: new Date("2026-08-10")
    },
    {
        user_id: 102,
        restaurant_id: 2,
        items: [
            { item_name: "Margherita Pizza", price: 350 },
            { item_name: "Garlic Bread", price: 150 }
        ],
        total_amount: 500,
        order_date: new Date("2026-08-11")
    },
    {
        user_id: 103,
        restaurant_id: 1,
        items: [
            { item_name: "Biryani", price: 300 },
            { item_name: "Cold Drink", price: 80 }
        ],
        total_amount: 380,
        order_date: new Date("2026-08-12")
    }
])
```

3. **Write a MongoDB aggregation query to calculate the total number of orders placed for each restaurant in the 'orders' collection and display the restaurant_id with its corresponding order count.

Hint: Use the $group stage in the aggregation pipeline.**

Ans:

Use $group with restaurant_id:

```
db.orders.aggregate([
    {
        $group: {
            _id: "$restaurant_id",
            orderCount: { $sum: 1 }
        }
    },
    {
        $project: {
            _id: 0,
            restaurant_id: "$_id",
            orderCount: 1
        }
    }
])
```

Example result:

{ restaurant_id: 1, orderCount: 2 }

{ restaurant_id: 2, orderCount: 1 }

4. **Generate a report using MongoDB aggregation that finds the top 2 restaurants with the highest average order amount from your 'orders' collection.

Hint: Use $group to calculate average, then $sort and $limit.**

Ans:

```
db.orders.aggregate([
    {
        $group: {
            _id: "$restaurant_id",
            averageOrderAmount: {
                $avg: "$total_amount"
            }
        }
    },
    {
        $sort: {
            averageOrderAmount: -1
        }
    },
    {
        $limit: 2
    },
    {
```

```
        $project: {
            _id: 0,
            restaurant_id: "$_id",
            averageOrderAmount: 1
        }
    }
])
```

5. **Refactor your 'orders' collection to include a 'delivery_status' field (values: 'pending', 'out for delivery', 'delivered'). Update all existing orders to randomly assign one of these statuses, then write a query to count how many orders are currently 'pending'.**

Ans:

```
db.orders.aggregate([
    {
        $set: {
            delivery_status: {
                $arrayElemAt: [
                    ["pending", "out for delivery", "delivered"],
                    { $floor: { $multiply: [{ $rand: {} }, 3] } }
                ]
            }
        }
    },
    {
        $merge: {
            into: "orders",
            on: "_id",
            whenMatched: "merge",
            whenNotMatched: "discard"
        }
    }
])
```

Verify:

db.orders.find().pretty()

You should see something like:

```
{
    user_id: 101,
    restaurant_id: 1,
    ...
    delivery_status: "delivered"
}
```

```
{
    user_id: 102,
    restaurant_id: 2,
    ...
    delivery_status: "pending"
}
{
    user_id: 103,
    restaurant_id: 1,
    ...
    delivery_status: "out for delivery"
}
```

Finally, count the currently pending orders:

```
db.orders.countDocuments({
    delivery_status: "pending"
})
```